CHAPTER 7

Applications of Deep Learning

LEARNING OBJECTIVES

After reading this chapter, you will be able to

- Understand how to approach a deep learning solution.
- Identify what kind of neural network can be used for a given problem.
- Get an exposure to practical applications of neural networks.

7.1 | INTRODUCTION

Deep learning algorithms are one of the ways of solving artificial intelligence problems (subfields) including Natural Language Processing (NLP), Automatic Speech Recognition (ASR), Human–Computer Interaction (HCI) and expert systems. Developing deep learning solutions for several problems require the following steps to be followed in sequence:

1. Data collection and cleaning
2. Pre-processing
3. Model building
4. Training and testing

Each phase is significant to obtain accurate results. This chapter explains some example problems in those subfields and how to implement them using deep learning frameworks discussed in Chapter 6.

7.2 | IMAGE CLASSIFICATION USING CNN

7.2.1 | Problem Description

Convolutional neural networks (CNNs) are very similar to ordinary neural networks. They are made up of neurons that have learnable weights and biases. Each neuron receives some inputs, performs a dot product and optionally follows it with a non-linear activations. CNN architectures make the

explicit assumption that the inputs are images, which allows us to encode certain properties into the architecture. These then make the forward function more efficient to implement and vastly reduce the number of parameters in the network.

We know that the machine's perception of an image is completely different from what we see. In fact, it is only numbers that machines see in an image. Each pixel in the image is given a value between 0 and 255. Thus, for the machine to classify any image, it requires some preprocessing for finding patterns or features that distinguish an image from another. The following is an image classification problem making use of the convolutional neural network in Keras. The dataset used here is CIFAR-10 which can be found at *https://www.cs.toronto.edu/~kriz/cifar.html*

7.2.2 | Language and Framework

Language used: Python
Framework used: Keras using TensorFlow backend and implemented on Jupyter Notebook, Anaconda IDE.

7.2.3 | Dataset

The CIFAR-10 dataset contains 60,000 32 × 32 color images in 10 classes, with 6,000 images per class. There are 50,000 training images and 10,000 test images. The dataset is divided into five training batches and one test batch, each with 10,000 images. The test batch contains exactly 1,000 randomly selected images from each class. The training batches contain the remaining images in random order, but some training batches may contain more images from one class than another. Between them, the training batches contain exactly 5,000 images from each class.

The classes are completely mutually exclusive. There is no overlap between automobiles and trucks. 'Automobile' includes sedans, SUVs and so on. 'Truck' includes only big trucks. 'Neither' includes pickup trucks.

Figure 7.1 shows the classes in the dataset as well as 10 random images from each.

7.2.4 | Code Snippets

These code snippets are based on Keras CNN-CIFAR/keras-cnn-cifar10.ipynb.

```
from keras.datasets import cifar10
(train_images, train_labels), (test_images, test_labels) =
cifar10.load_data()
```

Initially, we need to import the data. The CIFAR-10 dataset comes bundled with Keras. We will use the *load_data()* function to load it into memory.

```
classes = np.unique(train_labels)
no_of_classes = len(classes)
```



FIGURE 7.1 Labels in CIFAR-10 dataset.

Then we explore the data and find the number of images in training and test data and the number of classes. Once loading and exploring of data is completed, the data is preprocessed. Preprocessing of data includes normalization of data and converting the labels to categorical variables.

```
no_of_rows,no_of_cols,no_of_dims = train_im.shape[1:]
training_Data = train_im.reshape(train_im.shape[0], no_of_rows,
no_of_cols, no_of_dims)
testing_Data = test_im.reshape(test_im.shape[0], no_of_rows,
no_of_cols, no_of_dims)
input_shape = (no_of_rows, no_of_cols, no_of_dims)

training_Data = training_Data.astype('float32')
testing_Data = testing_Data.astype('float32')
```

```
training_Data /= 255
testing_Data /= 255

train_labels = to_categorical(train_lb)
test_labels = to_categorical(test_lb)
```

We find the shape of input images and create the variable *input_shape*. The datatype is changed to float and the data is scaled to lie between 0 and 1. The labels are converted to categorical data. Then we need to define the model. We can create the model with a convolutional layer, max pooling layer, dropout layer and dense layer. The first two layers are processed with 32 filters of window size 3 × 3.

```
def createModel():
      model = Sequential()
      model.add(Conv2D(32, (3, 3), padding='same',
      activation='relu',
      input_shape=input_shape))
      model.add(Conv2D(32, (3, 3), activation='relu'))
      model.add(MaxPooling2D(pool_size=(2, 2)))
      model.add(Dropout(0.25))

      model.add(Conv2D(64, (3, 3), padding='same',
      activation='relu'))
      model.add(Conv2D(64, (3, 3), activation='relu'))
      model.add(MaxPooling2D(pool_size=(2, 2)))
      model.add(Dropout(0.25))

      model.add(Conv2D(64, (3, 3), padding='same',
      activation='relu'))
      model.add(Conv2D(64, (3, 3), activation='relu'))
      model.add(MaxPooling2D(pool_size=(2, 2)))
      model.add(Dropout(0.25))

      model.add(Flatten())
      model.add(Dense(512, activation='relu'))
      model.add(Dropout(0.5))
      model.add(Dense(no_of_classes, activation='softmax'))

      return model
```

After defining the model, we need to train the model. Here the model is trained for 50 epochs with a batch size of 256.

```
model_one = createModel()
batch_size = 256
epochs = 50
model_one.compile(optimizer='rmsprop', loss=
'categorical_crossentropy',
metrics=['accuracy'])
model_one.summary()
history = model_one.fit(training_Data, train_labels,
batch_size=batch_size, epochs=epochs, verbose=1,validation_
data=(testing_Data, test_labels))
model_one.evaluate(test_data, test_labels)
```

The architecture of CNN used in this example is shown in Fig. 7.2.

Layer (type)	Output shape	Param #
conv2d_13 (Conv2D)	(None, 32, 32, 32)	896
conv2d_14 (Conv2D)	(None, 30, 30, 32)	9248
max_pooling2d_7 (MaxPooling2	(None, 15, 15, 32)	0
dropout_9 (Dropout)	(None, 15, 15, 32)	0
conv2d_15 (Conv2D)	(None, 15, 15, 64)	18496
conv2d_16 (Conv2D)	(None, 13, 13, 64)	36928
max_pooling2d_8 (MaxPooling2	(None, 6, 6, 64)	0
dropout_10 (Dropout)	(None, 6, 6, 64)	0
conv2d_17 (Conv2D)	(None, 6, 6, 64)	36928
conv2d_18 (Conv2D)	(None, 4, 4, 64)	36928
max_pooling2d_9 (MaxPooling2	(None, 2, 2, 64)	0
dropout_11 (Dropout)	(None, 2, 2, 64)	0
flatten_3 (Flatten)	(None, 256)	0
dense_4 (Dense)	(None, 512)	131584
dropout_12 (Dropout)	(None, 512)	0
dense_5 (Dense)	(None, 10)	5130

Total params: 276,138
Trainable params: 276,138
Non-trainable params: 0

FIGURE 7.2 Architecture of CNN used in the example.

7.2.5 | Complete Code

```
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline
from __future__ import print_function
import keras
from keras.models import Sequential
from keras.layers import Dense, Conv2D, MaxPooling2D, Drop-
out, Flatten

from keras.datasets import cifar10
(train_images, train_labels), (test_images, test_labels) =
cifar10.load_data()

from keras.utils import to_categorical

print('Shape of Training : ', train_images.shape,
train_labels.shape)
print('Shape of Testing data : ', test_images.shape,
test_labels.shape)

# Find the unique numbers from the train labels
classes = np.unique(train_labels)
no_of_Classes = len(classes)
print('Total number of outputs : ', no_of_Classes)
print('Output classes : ', classes)

"""Find the shape of input images and create the variable
input_shape"""
no_of_rows,no_of_cols,no_of_dims = train_images.shape[1:]
training_Data = train_images.reshape(train_images.shape[0], no_
of_rows, no_of_cols, no_of_dims)
testing_Data = test_images.reshape(test_images.shape[0], no_of_
rows, no_of_cols, no_of_dims)
input_shape = (no_of_rows, no_of_cols, no_of_dims)

# Change the datatype to float
training_Data = training_Data.astype('float32')
testing_Data = testing_Data.astype('float32')

# Scale the data to lie between 0 to 1
training_Data /= 255
testing_Data /= 255

# Change the labels from integer to categorical data
train_Labels = to_categorical(train_labels)
test_Labels = to_categorical(test_labels)
```

```
# Display the change for category label using one-hot encoding
print('Original label 0 : ', train_labels[0])
print('On conversion to categorical ( one-hot ) : ', train_
Labels[0])

def createModel():
    model = Sequential()
    # The first two layers with 32 filters of window size 3x3
    model.add(Conv2D(32, (3, 3), padding='same',
    activation='relu',
    input_shape=input_shape))
    model.add(Conv2D(32, (3, 3), activation='relu'))
    model.add(MaxPooling2D(pool_size=(2, 2)))
    model.add(Dropout(0.25))

    model.add(Conv2D(64, (3, 3), padding='same',
    activation='relu'))
    model.add(Conv2D(64, (3, 3), activation='relu'))
    model.add(MaxPooling2D(pool_size=(2, 2)))
    model.add(Dropout(0.25))

    model.add(Conv2D(64, (3, 3), padding='same',
    activation='relu'))
    model.add(Conv2D(64, (3, 3), activation='relu'))
    model.add(MaxPooling2D(pool_size=(2, 2)))
    model.add(Dropout(0.25))

    model.add(Flatten())
    model.add(Dense(512, activation='relu'))
    model.add(Dropout(0.5))
    model.add(Dense(no_of_Classes, activation='softmax'))

    return model

model_one = createModel()
batch_size = 3
epochs = 3
model_one.compile(optimizer='rmsprop',loss=
'categorical_crossentropy',
metrics=['accuracy'])
model_one.summary()
history = model_one.fit(training_Data, train_Labels, batch_
size=batch_size, epochs=epochs, verbose=1, validation_
data=(testing_Data, test_Labels))
model_one.evaluate(testing_Data, test_Labels)
```

```
model_one.predict(testing_Data)

# Loss Curves
plt.figure(figsize=[8,6])
plt.plot(history.history['loss'],'r',linewidth=3.0)
plt.plot(history.history['val_loss'],'b',linewidth=3.0)
plt.legend(['Training loss', 'Validation Loss'],fontsize=18)
plt.xlabel('Epochs',fontsize=16)
plt.ylabel('Loss',fontsize=16)
plt.title('Loss Curves',fontsize=16)

# Accuracy Curves
plt.figure(figsize=[8,6])
plt.plot(history.history['acc'],'r',linewidth=3.0)
plt.plot(history.history['val_acc'],'b',linewidth=3.0)
plt.legend(['Training Accuracy', 'Validation
Accuracy'],fontsize=18)
plt.xlabel('Epochs',fontsize=16)
plt.ylabel('Accuracy',fontsize=16)
plt.title('Accuracy Curves',fontsize=16)
```

7.2.6 | Result

Each value in the array represents the category of that particular image in the test dataset. Figure 7.3 represents the output results and Fig. 7.4 represents the loss and accuracy curves.

```
array( [[3 .4562217e-03,  9.2505850e-04,  1.8809618e-02,  …,  8.5772546e-03,
         7.6413718e-03,  3.8437799e-04],
        [2.4021919e-01,  3.6076340e-01,  3.5939459e-03,  …,  3.2659632e-04,
         3.5520118e-01,  3.5589766e-02],
        [2.6662815e-01,  5.5819660e-02,  1.6485509e-01,  …,  1.3124739e-02,
         2.5609285e-01, 3.0395191e-02],
        …,
        [1.8850275e-03,  1.5383511e-04,  6.3279994e-02,  …,  5.5726457e-02,
         4.1047897e-04,  3.2096161e-04],
        [5.3246684e-02,  5.1997840e-02,  6.4273000e-02,  …,  7.0452951e-02,
         2.2547310e-03,  3.1353589e-03],
        [3.5260079e-04,  9.4345105e-06,  3.8702232e-03,  …,  8.7736481e-01,
         1.4745045e-06,  1.3149938e-05]],  dtype=float 32)
```

FIGURE 7.3 Integer values of predicted classes.

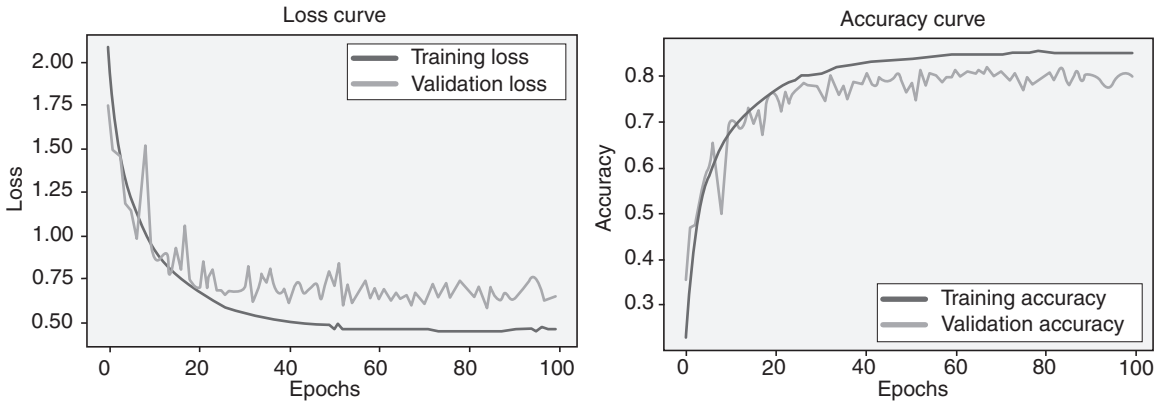


FIGURE 7.4 Loss and accuracy curves.

7.3 | VISUAL SPEECH RECOGNITION USING 3D-CNN

7.3.1 | Problem Description

Let us consider the following scenario: A person is mute and he wants to communicate. Just imagine if there is a machine that visualizes his lips movements and converts it into text! Visual speech recognition aims at transcribing lip movements into readable text. The system will convert an input video into transcribed text. The technique uses classification of visemes into texts; visemes are particular lip moments that can be converted into text. In recent years, there have been many advances in automatic speech reading system with the inclusion of audio and visual speech features to recognize words even under noisy conditions. The proposed system focuses mainly on the visual element as a support for the acoustic segment. It classifies a set of visemes to text. The system has two modules. The first one handles extraction of lip features from input while the second is a neural network system trained to process the lip visemes and classify into text.

7.3.2 | Language and Framework

Language used: Python
Framework used: Keras using TensorFlow backend and implemented on Spyder, Anaconda IDE.

7.3.3 | Dataset

MIRACL-VC1 is a lip-reading dataset that has both depth and color images. It can be used for diverse research fields like visual speech recognition, face detection and biometrics. Here, 15 speakers (5 men and 10 women) positioned in the frustum of an MS Kinect sensor have uttered a set of 10 words and 10 phrases, 10 times. Each instance of the dataset consists of a synchronized sequence of color and depth images (both of 640 × 480 pixels). The MIRACL-VC1 dataset contains a total number of 3,000 instances (15 × 10 × 10). In the proposed system, only the color images are used. The sample color and depth image frames from MIRACL-VC1 dataset is shown in Fig. 7.5. This example uses only the words from the dataset.